

IPCA



**INSTITUTO POLITÉCNICO
DO CÁVADO E DO AVE
ESCOLA SUPERIOR
DE TECNOLOGIA**

**Instituto Politécnico do Cávado e do Ave
Escola Superior de Tecnologia**

**Licenciatura em
Engenharia de Sistemas Informáticos**

Trabalho Prático (Fase 2)

Fábio Rafael Gomes Costa

Barcelos, maio de 2025

**Instituto Politécnico do Cávado e do Ave
Escola Superior de Tecnologia**

Licenciatura em Engenharia de Sistemas Informáticos

Trabalho Prático (Fase 2)

Fábio Rafael Gomes Costa – a22997

Unidade Curricular:

Estruturas de Dados Avançadas

Docente:

Dr. Luís Gonzaga Martins Ferreira

Barcelos, maio de 2025

Ficha de Identificação

Elaborado por Fábio Rafael Gomes Costa

Contato a22997@alunos.ipca.pt

Unidade Curricular Estruturas de Dados Avançadas

Curso Licenciatura em Engenharia de Sistemas Informáticos

Instituição Escola Superior de Tecnologia do
Instituto Politécnico do Cávado e do Ave

Professor Doutor Luís Gonzaga Martins Ferreira

Gabinete 5

Contato lufer@ipca.pt

Data de início 5 de maio de 2025

Data de conclusão 18 de maio de 2025

Resumo

O presente trabalho insere-se no âmbito da Unidade Curricular de Estruturas de Dados Avançadas (EDA) e tem como objetivo principal a aplicação prática dos conhecimentos adquiridos, através do desenvolvimento de uma solução que permita representar e manipular dados referentes a antenas e suas localizações numa cidade, utilizando estruturas de dados dinâmicas baseadas em grafos, implementadas em linguagem C.

Na Fase 2 do projeto, a estrutura de dados foi reformulada, substituindo as listas ligadas simples da fase anterior por um grafo representado por listas de adjacência, onde cada vértice corresponde a uma antena e cada aresta a uma ligação entre antenas de mesma frequência. Além disso, foi incorporada a lógica para armazenar as zonas nefastas diretamente nas arestas, sempre que duas antenas criassem interferência com base na sua posição espacial.

Foram implementados algoritmos clássicos de travessia em profundidade (DFS) e largura (BFS), assim como a funcionalidade para determinar todos os caminhos possíveis entre duas antenas. A construção e manipulação do grafo permitiram aprofundar o conhecimento sobre estruturas ligadas, algoritmos de grafos e análise de padrões espaciais.

O desenvolvimento da fase decorreu num ambiente controlado, com recurso ao Visual Studio, e todo o projeto foi registado através da plataforma GitHub. A documentação técnica foi produzida com o auxílio da ferramenta Doxygen, assegurando a clareza e organização do código-fonte.

Índice

Ficha de Identificação.....	I
Resumo	II
Índice	III
Índice de Fragmentos de Código.....	IV
Índice de Figuras	V
Lista de Siglas e Acrônimo.....	V
1. Introdução.....	1
1.1 Contextualização	1
1.2 Motivação/Pretensões e Objetivos	1
1.3 Metodologia de Trabalho.....	2
1.4 Ferramentas e Tecnologias Utilizadas.....	2
1.5 Estrutura do Documento	2
2. Fundamentos Teóricos	4
2.1 Listas Ligadas.....	4
2.2 Grafos e Listas de Adjacência	4
2.3 Algoritmos de Travessia (DFS e BFS)	5
2.4 Backtracking para todos os caminhos	5
3. Problemas.....	6
4. Resolução dos Problemas.....	10
4.1 Estrutura do grafo (Problema 1)	10
4.1.1 Antenas/Vértices.....	10
4.1.2 Arestas e Zonas Nefastas.....	10
4.1.3 Grafo.....	11
4.2 Carregamento de dados (Problema 2)	12
4.2.1 Função LerLista.....	12

4.2.2	Função InserirAntena	13
4.2.3	Função CriarListaArestas	14
4.3	Procura em profundidade DFS (Problema 3.a.)	17
4.3.1	Função DFS	17
4.3.2	Função IniciarDFS.....	18
4.4	Procura em largura BFS (Problema 3.b.)	19
4.4.1	Função BFS	19
4.5	Caminhos possíveis entre antena (Problemas 3.c.).....	21
4.5.1	Função EncontrarCaminhos	21
4.5.2	Função TodosCaminhos	22
5.	Repositório GitHub	23
6.	Considerações Finais.....	24
7.	Referencias bibliográfica.....	25

Índice de Fragmentos de Código

Código 1 - struct Ant	10
Código 2 - struct Ars	11
Código 3 - struct Grafo	11
Código 4 - Função LerLista.....	13
Código 5 - Função InserirAntena	14
Código 6 - Função CriarListaArestas	16
Código 7 - Função DFS	18
Código 8 - Função IniciarDFS	18
Código 9 - Função BFS	20
Código 10 - Função EncontrarCaminhos	21
Código 11 - Função TodosCaminhos	22

Índice de Figuras

Figura 1 - Matriz Problema I	6
Figura 2 - Matriz Problema II.....	7
Figura 3 - Matriz Problema III	7
Figura 4 - Matriz Problema IV	8
Figura 5 - Matriz Problema V.....	9
Figura 6 - Arestas e Nefastos.....	11
Figura 7 - DFS – Depth-First Search.....	17
Figura 8 - BFS – Breadth-First Search	19

Lista de Siglas e Acrônimo

EDA – Estruturas de Dados Avançadas

ID – Identificador (Identificador Único)

IPCA – Instituto Politécnico do Cavado e do Ave

UC – Unidade Curricular

VS – Visual Studio 2022

Ant – Antena

Nef – Nefasto

Ars – Aresta

DFS – Depth-First Search

BFS – *Breadth-First Search*

1. Introdução

1.1 Contextualização

No âmbito da Unidade Curricular (UC) de Estruturas de Dados Avançadas (EDA), inserida no 2º semestre do 1º ano do curso, foi-me atribuída a realização de um projeto prático como instrumento de avaliação, sob a orientação do docente Dr. Luís G. Ferreira. Este projeto tem como principal objetivo a aplicação e consolidação dos conhecimentos adquiridos ao longo do semestre, através da implementação e manipulação de estruturas de dados dinâmicas na linguagem de programação C.

1.2 Motivação/Pretensões e Objetivos

O desenvolvimento da Fase 2 deste projeto tem como principal motivação o aprofundamento do estudo e aplicação de estruturas de dados do tipo grafo, uma das mais relevantes na modelação de sistemas complexos. Pretende-se consolidar o conhecimento sobre a representação de grafos por listas de adjacência, integrando esse modelo com conceitos previamente abordados, como listas ligadas e modularização de código.

Os principais objetivos desta fase são:

- Implementar algoritmos clássicos de travessia, como DFS (Depth-First Search) e BFS (Breadth-First Search);
- Desenvolver mecanismos para identificar todos os caminhos possíveis entre duas antenas, recorrendo a backtracking;
- Reforçar a capacidade de resolução de problemas de conectividade, acessibilidade e propagação em redes.

Adicionalmente, esta fase promove boas práticas de desenvolvimento, como:

- Organização modular do código;
- Documentação técnica com Doxygen;
- Utilização eficiente da memória através de vetores e estruturas dinâmicas;
- Aplicação de estruturas e algoritmos a dados espaciais em formato matricial.

1.3 Metodologia de Trabalho

Nesta segunda fase, foi mantida uma abordagem faseada e incremental, dando continuidade à estrutura desenvolvida anteriormente. Inicialmente, procedeu-se à análise do enunciado da fase 2, com o objetivo de identificar as novas exigências técnicas, nomeadamente a representação do grafo, o cálculo das zonas nefastas, e a implementação de algoritmos de percursos.

O código foi organizado de forma modular e extensível, garantindo compatibilidade com as estruturas já existentes. A implementação do grafo por lista de adjacência foi integrada diretamente à estrutura das antenas, permitindo uma representação eficiente das conexões entre vértices.

Os algoritmos de DFS, BFS e backtracking foram desenvolvidos individualmente, testados de forma isolada, e depois integrados no sistema global. Durante o processo, foram realizados testes frequentes para garantir o correto funcionamento das travessias e a coerência dos caminhos identificados.

A documentação foi atualizada com o auxílio do Doxygen, mantendo a clareza da arquitetura do código e facilitando a compreensão das funções envolvidas. Todo o desenvolvimento foi acompanhado com controlo de versões através do GitHub.

1.4 Ferramentas e Tecnologias Utilizadas

No desenvolvimento do projeto, utilizei a linguagem de programação C, utilizada no contexto de sala de aula. O *Visual Studio* foi o ambiente escolhido pela sua interface intuitiva e funcionalidades de depuração. Para o controlo de versões, recorreu-se ao *GitHub*, garantindo o acompanhamento e preservação do código. A documentação foi gerada automaticamente com o *Doxygen*, a partir de comentários no código. Finalmente, o *Microsoft Word* foi utilizado para a redação do relatório.

1.5 Estrutura do Documento

Este relatório foi organizado de forma a apresentar o trabalho desenvolvido de maneira clara e sequencial. Inicia-se com a capa, subcapa e Ficha de Identificação, seguida

do Resumo e do Índice, que inclui tanto os tópicos principais como o Índice de Figuras, o Índice de Fragmentos de Código e a Lista de Siglas e Acrónimos.

A Introdução apresenta o enquadramento geral do projeto, incluindo a contextualização, os objetivos, a metodologia e as ferramentas utilizadas.

Seguindo a introdução, encontra-se o Enquadramento Teórico e Prático, onde são discutidos os fundamentos teóricos.

Na secção seguinte, Problemas, são identificados e descritos os problemas enfrentados durante o desenvolvimento do projeto.

O Desenvolvimento do projeto está dividido em duas fases: a resolução dos problemas antes da defesa, detalhada na Resolução dos Problemas (Antes da Defesa), e as modificações feitas após a defesa, descritas em Resolução dos Problemas (Pós-Defesa). Cada problema é analisado individualmente, com as respetivas soluções apresentadas nas subsecções de Resolução do Problema 1, Resolução do Problema 2, e assim por diante, até o Problema 3.d.

A seguir, encontra-se o Repositório GitHub, onde é apresentado o endereço do repositório do projeto.

O relatório é concluído com as Considerações Finais, onde são apresentadas as reflexões sobre o trabalho realizado, as aprendizagens obtidas e sugestões de melhoria.

Por fim, são apresentadas as Referências Bibliográficas, onde se encontram listados todos os links consultados durante o desenvolvimento do projeto.

2. Fundamentos Teóricos

Este projeto assenta em vários conceitos fundamentais da teoria das estruturas de dados e algoritmos, com especial enfoque na sua aplicação prática à representação e análise de redes de antenas. A seguir, são descritos os principais fundamentos teóricos utilizados e como foram aplicados no desenvolvimento da solução.

2.1 Listas Ligadas

As listas ligadas são uma estrutura dinâmica fundamental onde cada elemento (ou nó) aponta para o próximo, permitindo uma gestão flexível da memória. No contexto deste projeto, cada antena foi inicialmente armazenada numa lista ligada, o que facilitou:

- A inserção dinâmica de antenas durante a leitura da matriz;
- A criação de uma estrutura simples e sequencial para iteração;
- A separação lógica entre as antenas e as suas relações (arestas).

Esta estrutura foi essencial na fase 1 e serviu de base para a construção do grafo na fase seguinte.

2.2 Grafos e Listas de Adjacência

Um grafo é uma estrutura matemática composta por vértices e arestas, utilizada para modelar relações entre entidades. No projeto, as antenas foram tratadas como vértices e as ligações entre antenas com a mesma frequência como arestas.

Foi utilizada a técnica da lista de adjacência, em que cada vértice (antena) mantém uma lista de arestas que apontam para os seus vizinhos. Esta escolha permitiu:

- Representar o grafo de forma mais eficiente em termos de memória, especialmente em grafos esparsos;
- Acessar diretamente os vizinhos de uma antena para percursos e análises;
- Associar dados adicionais às arestas, como as coordenadas das zonas nefastas causadas pela interferência.

Esta abordagem reflete um modelo prático de representação de redes reais, como redes de comunicação ou infraestruturas urbanas.

2.3 Algoritmos de Travessia (DFS e BFS)

Para explorar as conexões dentro do grafo, foram implementados dois algoritmos de travessia fundamentais:

- DFS (Depth-First Search): Segue um caminho até o fim antes de recuar. Foi usado para:
 - Verificar quais antenas podem ser alcançadas a partir de uma origem;
 - Explorar componentes conexas do grafo;
 - Gerar percursos profundos para análise.
- BFS (Breadth-First Search): Explora todos os vizinhos antes de aprofundar. Foi usado para:
 - Listar as antenas acessíveis por camadas de distância;
 - Simular alcançabilidade em rede em saltos mínimos;
 - Representar percursos mínimos em termos de número de arestas.

Ambos os algoritmos foram adaptados para registrar e imprimir os caminhos percorridos, respeitando a estrutura da lista de adjacência.

2.4 Backtracking para todos os caminhos

Para listar todos os caminhos possíveis entre duas antenas, foi aplicado o conceito de backtracking, uma técnica recursiva que permite explorar todas as soluções possíveis num grafo. Essa abordagem foi utilizada para:

- Percorrer o grafo sem repetições, utilizando um vetor de visitados;
- Armazenar e imprimir os caminhos válidos;
- Analisar a redundância e conectividade múltipla entre antenas.

Este conjunto de conceitos e algoritmos permitiu não só a resolução dos problemas propostos, mas também a aproximação prática entre a teoria das estruturas de dados e a sua aplicabilidade em contextos reais, como análise de redes, comunicação e representação espacial.

3. Problemas

O presente capítulo serve para apresentar ao leitor os problemas utilizados como base para a resolução do projeto.

“O seguinte texto e as seguintes imagens foram retiradas do Enunciado do trabalho pratico disponibilizado pelo docente na plataforma académica Moodle, para quem tiver interesse de consultar o documento na integra, esta disponibilizada uma copia no repositório do *GitHub* do projeto.”

Pretende-se considerar uma cidade com várias antenas. Cada antena é sintonizada numa frequência específica indicada por um caracter. O mapa das antenas com as suas localizações (coordenadas na matriz) e frequências é representado através de uma matriz. Por exemplo:

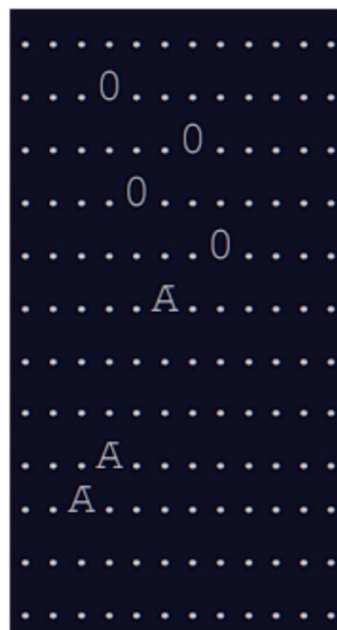


Figura 1 - Matriz Problema I

Considere que o sinal de cada antena aplica um efeito nefasto em localizações específicas L com base nas frequências de ressonância das antenas. Em particular, o efeito nefasto ocorre em qualquer localização L que esteja perfeitamente alinhada com duas antenas da mesma frequência - mas apenas quando uma das antenas está duas vezes mais

distante que a outra. Isso significa que para qualquer par de antenas com a mesma frequência, existem duas localizações, uma de cada lado das antenas.

A título de exemplo, para duas antenas com frequência a, as localizações com efeito nefasto encontram-se representadas com #:

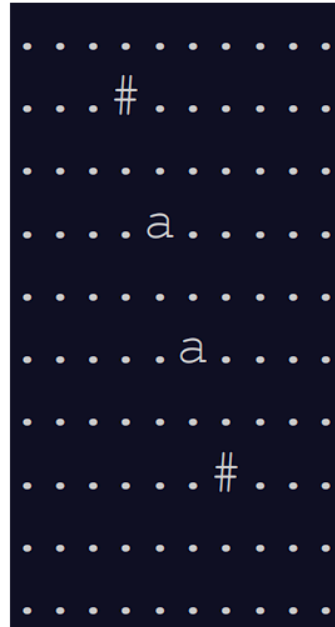


Figura 2 - Matriz Problema II

Adicionar uma terceira antena com a mesma frequência cria várias localizações adicionais com efeito nefasto:

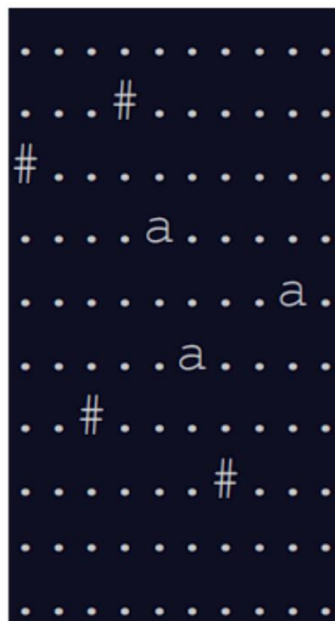


Figura 3 - Matriz Problema III

Antenas com frequências diferentes (caracteres diferentes) não criam localizações com efeito nefasto.

Localizações com efeito nefasto podem ocorrer em locais que contêm antenas. Uma localização com efeito nefasto pode surgir na sequência das várias combinações de antenas em simultâneo.

O primeiro exemplo tem antenas com duas frequências diferentes (A e O), dando origem as localizações com efeito nefasto seguintes:

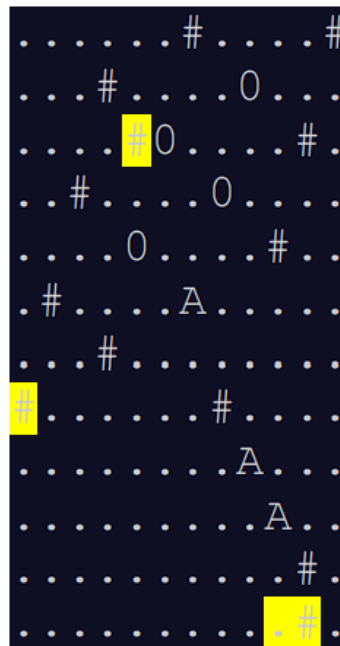


Figura 4 - Matriz Problema IV

Fase 2 - Grafos

Na segunda fase, pretende-se aplicar conceitos avançados de teoria dos grafos e programação em C para resolver o mesmo problema computacional.

Assim, procure implementar as funcionalidades seguintes:

1. *Definir um tipo de dados GR para representação de grafos, permitindo suportar um número qualquer de vértices. Cada vértice deverá representar uma antena e a sua localização numa cidade. As arestas deverão interligar somente antenas com frequências de ressonância iguais.*

2. Carregamento dos dados referentes às antenas de uma cidade, constantes num ficheiro de texto, devendo dar origem a um grafo com tipo de dados GR. A título de exemplo, o ficheiro de texto deverá respeitar o formato seguinte:

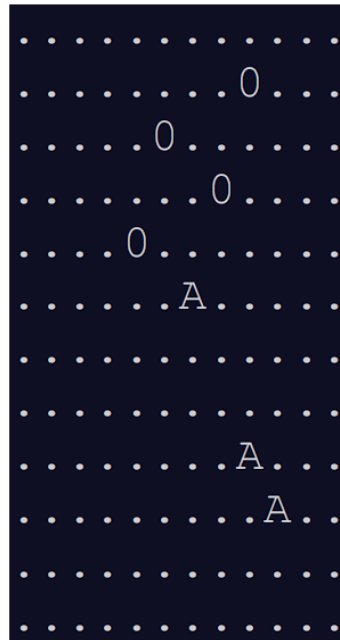


Figura 5 - Matriz Problema V

3. Implementar operações de manipulação de grafos do tipo GR, incluindo:
- Procura em profundidade a partir de uma determinada antena, listando na consola as coordenadas das antenas alcançadas;
 - Procura em largura a partir de uma determinada antena, listando na consola as coordenadas das antenas alcançadas;
 - Identificar todos os caminhos possíveis entre duas determinadas antenas, listando na consola as sequências de arestas interligando as antenas;
 - Dadas duas frequências de ressonância distintas A e B, listar na consola todas as intersecções de pares de antenas com ressonância A e B, indicando as respetivas coordenadas.

4. Resolução dos Problemas

4.1 Estrutura do grafo (Problema 1)

4.1.1 Antenas/Vértices

A estrutura utilizada para representar uma antena (ou vértice) na fase 2 mantém a base da estrutura da fase 1 do projeto, onde já se armazenavam informações como:

- a frequência da antena,
- o seu identificador único,
- as suas coordenadas na matriz,
- e o ponteiro para a próxima antena na lista ligada.

Contudo, nesta fase foi necessário adaptar a estrutura para representar corretamente um grafo com lista de adjacência. Assim, adicionou-se um campo adicional que aponta para a lista de arestas associadas a essa antena, permitindo armazenar todas as ligações (ou conexões) com outras antenas de frequência igual.

```
typedef struct Ant {  
    char freqAntena;  
    int id;  
    int x;  
    int y;  
    struct Ant* proxAntena;  
    struct Ars* listaAresta;  
} Ant;
```

Código 1 - struct Ant

4.1.2 Arestas e Zonas Nefastas

Uma aresta representa a ligação entre duas antenas com a mesma frequência. Como uma antena (origem) pode ligar-se a múltiplas outras antenas (destinos), é necessário utilizar uma lista de arestas para cada vértice (antena). Cada elemento dessa lista guarda:

- O endereço da antena de origem;
- O endereço da antena de destino;
- O endereço da próxima aresta (na lista ligada);
- E, adicionalmente, as coordenadas da zona nefasta (caso exista).

A estrutura utilizada foi a seguinte:

```
typedef struct Ars {
    struct Ant* origemAntena;
    struct Ant* destinoAntena;
    struct Ars* proximaAresta;
    int xNef;
    int yNef;
}Ars;
```

Código 2 - struct Ars

As zonas nefastas são determinadas geometricamente com base na posição relativa de duas antenas com a mesma frequência. Como o grafo é considerado bidirecional e fortemente ligado, a informação relativa à uma zona nefasta pode ser armazenada juntamente com a aresta, permitindo associar diretamente o efeito nefasto ao par de antenas responsável.

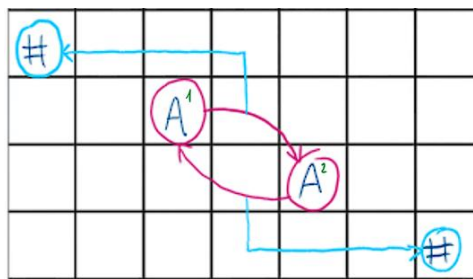


Figura 6 - Arestas e Nefastos

4.1.3 Grafo

A estrutura principal do grafo é responsável por armazenar os ponteiros para todas as antenas existentes. Esta organização é feita através de um array estático, onde o índice do array corresponde diretamente ao identificador único (ID) da antena.

Dessa forma, o acesso a qualquer antena é feito em tempo constante ($O(1)$), facilitando as operações de travessia e manipulação do grafo.

A estrutura é definida da seguinte forma:

```
typedef struct Grafo {
    struct Ant* Antena[MAX_VERTICES];
}Grafo;
```

Código 3 - struct Grafo

Com esta implementação, por exemplo, `grafo->Antena[1]` guarda o endereço da antena com `id = 1`, permitindo um acesso direto e eficiente a qualquer vértice do grafo.

4.2 Carregamento de dados (Problema 2)

Para o carregamento de dados, foram reaproveitadas e adaptadas algumas das funções desenvolvidas na Fase 1 do projeto. Estas funções foram ajustadas para suportar a nova estrutura baseada em grafos, assegurando a criação correta dos vértices (antenas), bem como das arestas que os interligam.

As principais funções utilizadas para o levantamento e estruturação dos dados foram:

- LerLista() – Responsável por ler a matriz de entrada a partir de um ficheiro e construir a estrutura de antenas;
- InserirAntena() – Insere dinamicamente cada antena lida na lista ligada e no grafo;
- CriarListaArestas() – Gera as ligações entre antenas com base na frequência, calculando também as zonas nefastas associadas.

4.2.1 Função LerLista

A função LerLista() tem como principal objetivo realizar a leitura do ficheiro que representa a matriz da cidade, onde se encontram as localizações das antenas. Esta função suporta ficheiros de texto (.txt) ou binários (.bin), sendo o tipo definido pelo argumento tipoFicheiro.

Durante a leitura:

- É construída uma lista ligada com todas as antenas encontradas na matriz;
- Cada antena é também registada no grafo, associando o seu identificador à respetiva posição no array de vértices;
- Ao final da leitura, é chamada a função CriarListaArestas() para gerar as arestas entre as antenas com a mesma frequência e calcular as zonas nefastas resultantes.

A implementação da função é apresentada abaixo:

```
Ant* LerLista(const char* nomeFicheiro, const char* tipoFicheiro, int* linhas, int*  
colunas, Grafo* grafo) {  
    FILE* ficheiro;  
    char nomeCompleto[256];  
    snprintf(nomeCompleto, sizeof(nomeCompleto), "%s%s", nomeFicheiro, tipoFicheiro);  
    if (strcmp(tipoFicheiro, ".txt") == 0) {  
        errno_t err = fopen_s(&ficheiro, nomeCompleto, "r");
```

```
        if (err != 0) {
            perror("Erro ao abrir o ficheiro");
            return NULL;
        }
    }
    else {
        errno_t err = fopen_s(&ficheiro, nomeCompleto, "rb");
        if (err != 0) {
            perror("Erro ao abrir o ficheiro");
            return NULL;
        }
    }
    memset(grafo->Antena, 0, sizeof(grafo->Antena));
    Ant* lista = NULL;
    char linha[MAX_COLUNAS];
    int y = 0;
    int maxColunas = 0;
    int id = 0;

    while (fgets(linha, sizeof(linha), ficheiro)) {
        int tamLinha = strlen(linha);
        if (linha[tamLinha - 1] == '\n') {
            linha[tamLinha - 1] = '\0';
            tamLinha--;
        }
        if (tamLinha > maxColunas) {
            maxColunas = tamLinha;
        }
        for (int x = 0; x < tamLinha; x++) {
            if (linha[x] != '.') {
                lista = InserirAntena(lista, grafo, linha[x], x, y, id);
                id++;
            }
        }
        y++;
    }
    *linhas = y;
    *colunas = maxColunas;
    int verificaArestas = CriarListaArestas(lista, y, maxColunas);
    if (verificaArestas == 0) {
        perror("Erro ao criar lista de arestas");
        fclose(ficheiro);
        return NULL;
    }
    fclose(ficheiro);
    return lista;
}
```

Código 4 - Função LerLista

4.2.2 Função InserirAntena

A função `InserirAntena()` é responsável por criar e inserir dinamicamente uma nova antena tanto na lista ligada de antenas como na estrutura de grafo. A função é chamada durante a leitura da matriz (`LerLista()`), sempre que é detetado um caracter diferente de ponto (`.`), que representa a presença de uma antena.

A função executa os seguintes passos:

- Aloca memória dinamicamente para uma nova antena.
- Preenche os campos da antena com os dados fornecidos: frequência, coordenadas, identificador.
- Insere a nova antena no início da lista ligada (lista), utilizando o padrão clássico de inserção à cabeça.
- Regista o ponteiro da antena no array do grafo (grafo->Antena[id]), associando o ID da antena à sua posição no grafo.

A implementação é apresentada abaixo:

```
Ant* InserirAntena(Ant* lista, Grafo* grafo, char freq, int x, int y, int id) {
    Ant* novaAntena = (Ant*)malloc(sizeof(Ant));
    if (novaAntena == NULL) {
        perror("Erro ao alocar memória para antena");
        return lista;
    }
    novaAntena->freqAntena = freq;
    novaAntena->x = x;
    novaAntena->y = y;
    novaAntena->id = id;
    novaAntena->proxAntena = lista;
    novaAntena->listaAresta = NULL;
    if (grafo != NULL && id < MAX_VERTICES) {
        grafo->Antena[id] = novaAntena;
    }
    return novaAntena;
}
```

Código 5 - Função InserirAntena

Esta construção inicial da estrutura, permite que cada antena esteja imediatamente acessível tanto por lista (apenas utilizado para identificar os nefastos e as arestas) como por índice no grafo, facilitando operações futuras como travessias, buscas e ligações.

4.2.3 Função CriarListaArestas

A função CriarListaArestas() é responsável por gerar as ligações (arestas) entre antenas com a mesma frequência, representando o grafo através de listas de adjacência.

Para cada par de antenas diferentes (com coordenadas distintas), é verificada a coincidência de frequência. Se ambas partilharem a mesma frequência, é criada uma aresta

da antena de origem para a antena de destino. Esta aresta é inserida na lista de adjacência da antena de origem.

Além da ligação entre antenas, esta função também calcula, com base nas posições espaciais das antenas envolvidas, a possível zona nefasta. A localização dessa zona é calculada geometricamente, assumindo-se que ela ocorre por simetria em relação à linha formada pelas duas antenas envolvidas.

```
int CriarListaArestas(Ant* lista, int linhas, int colunas) {
    int menorx, menory, maiorx, maiory, difx, dify, idant1, idant2;
    Ant* listaAnt1 = lista;
    while (listaAnt1 != NULL) {
        Ant* listaAnt2 = lista;
        while (listaAnt2 != NULL) {
            if (listaAnt1->freqAntena == listaAnt2->freqAntena && (listaAnt1->y !=
listaAnt2->y || listaAnt1->x != listaAnt2->x)) {
                Ars* novaAresta = (Ars*)malloc(sizeof(Ars));
                if (novaAresta == NULL) {
                    perror("Erro ao alocar memoria para a aresta");
                    return 500;
                }
                novaAresta->origemAntena = listaAnt1;
                novaAresta->destinoAntena = listaAnt2;
                novaAresta->proximaAresta = listaAnt1->listaAresta;
                listaAnt1->listaAresta = novaAresta;
                if (listaAnt1->y != listaAnt2->y || listaAnt1->x != listaAnt2->x){
                    if (listaAnt1->x > listaAnt2->x)
                    {
                        difx = listaAnt1->x - listaAnt2->x;
                        menorx = listaAnt2->x - difx;
                        maiorx = listaAnt1->x + difx;
                    }
                    else
                    {
                        difx = listaAnt2->x - listaAnt1->x;
                        menorx = listaAnt1->x - difx;
                        maiorx = listaAnt2->x + difx;
                    }
                }
                if (listaAnt1->y > listaAnt2->y)
                {
                    dify = listaAnt1->y - listaAnt2->y;
                    menory = listaAnt2->y - dify;
                    maiory = listaAnt1->y + dify;
                }
                else
                {
                    dify = listaAnt2->y - listaAnt1->y;
                    menory = listaAnt1->y - dify;
                    maiory = listaAnt2->y + dify;
                }
                idant1 = listaAnt1->id;
                idant2 = listaAnt2->id;
                if (listaAnt1->x > listaAnt2->x && listaAnt1->y > listaAnt2->y ||
listaAnt1->x < listaAnt2->x && listaAnt1->y < listaAnt2->y)
                {
                    if (listaAnt1->y < listaAnt2->y) {
```



```
        if (menorx >= 0 && menory >= 0 && menorx <= linhas && menory
<= colunas)
        {
            novaAresta->xNef = menorx;
            novaAresta->yNef = menory;
        }
    }
    else {
        if (maiorx <= colunas && maiory <= linhas && maiorx <=
linhas && maiory <= colunas)
        {
            novaAresta->xNef = maiorx;
            novaAresta->yNef = maiory;
        }
    }
    else if (listaAnt1->x > listaAnt2->x && listaAnt1->y < listaAnt2->y
|| listaAnt1->x < listaAnt2->x && listaAnt1->y > listaAnt2->y)
    {
        if (listaAnt1->y < listaAnt2->y) {
            if (maiorx >= 0 && menory >= 0 && maiorx <= linhas && menory
<= colunas)
            {
                novaAresta->xNef = maiorx;
                novaAresta->yNef = menory;
            }
        }
        else {
            if (menorx <= colunas && maiory <= linhas && menorx <=
linhas && maiory <= colunas)
            {
                novaAresta->xNef = menorx;
                novaAresta->yNef = maiory;
            }
        }
    }
}
listaAnt2 = listaAnt2->proxAntena;
}
listaAnt1 = listaAnt1->proxAntena;
}
return 200;
}
```

Código 6 - Função CriarListaArestas

4.3 Procura em profundidade DFS (Problema 3.a.)

A procura em profundidade (DFS – Depth-First Search) é um algoritmo clássico de travessia de grafos. A sua principal característica é a exploração do grafo o mais profundamente possível antes de retroceder. Neste projeto, a DFS foi utilizada para listar todas as antenas que podem ser alcançadas a partir de uma antena de origem, explorando as suas conexões diretas e indiretas através das arestas definidas na lista de adjacência.

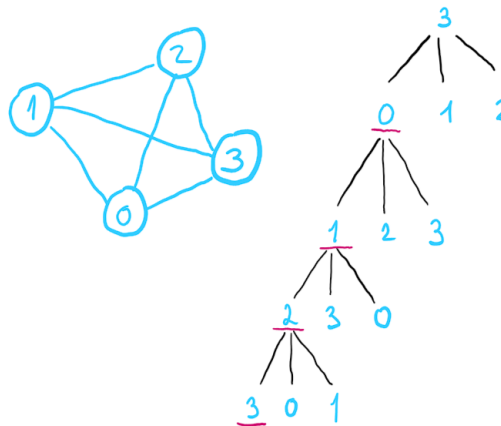


Figura 7 - DFS – Depth-First Search

Este algoritmo é útil, por exemplo, para verificar se todas as antenas de uma mesma frequência estão ligadas, ou para detetar componentes conexas no grafo.

4.3.1 Função DFS

A função DFS() realiza a travessia recursiva do grafo. Para evitar ciclos e repetições, é utilizado um vetor visitado[], que regista os identificadores das antenas já processadas. A função imprime as coordenadas e a frequência de cada antena visitada, bem como a ligação que levou até ela, exceto no caso da antena de origem (marcada com idOrigem == -1).

```
int DFS(Ant* atual, int visitado[], int idOrigem) {
    if (atual == NULL || visitado[atual->id]) {
        return 200;
    }
    visitado[atual->id] = 1;
    if (idOrigem == -1) {
        printf("Antena %d (%c) em [%d, %d] (Origem)\n",
            atual->id, atual->freqAntena, atual->x, atual->y);
    }
    else {
        printf("Antena %d (%c) em [%d, %d] (Aresta: origem %d -> destino %d)\n",
            atual->id, atual->freqAntena, atual->x, atual->y, idOrigem, atual->id);
    }
}
```

```
    }  
    Ars* aresta = atual->listaAresta;  
    while (aresta != NULL) {  
        DFS(aresta->destinoAntena, visitado, atual->id);  
        aresta = aresta->proximaAresta;  
    }  
    return 200;  
}
```

Código 7 - Função DFS

4.3.2 Função IniciarDFS

A função IniciarDFS() é responsável por inicializar a travessia em profundidade, verificando previamente se a antena de origem existe no grafo. Em caso afirmativo, inicializa o vetor de visitados e chama a função recursiva DFS() com o ID da antena fornecida.

```
int IniciarDFS(Grafo* grafo, int idOrigem) {  
    if (idOrigem < 0 || idOrigem >= MAX_VERTICES || grafo->Antena[idOrigem] == NULL) {  
        fprintf(stderr, "Antena com ID %d não existe.\n", idOrigem);  
        return 500;  
    }  
    int visitado[MAX_VERTICES] = { 0 };  
    printf("DFS a partir da antena %d:\n", idOrigem);  
    DFS(grafo->Antena[idOrigem], visitado, -1);  
    printf("\n\n");  
    return 200;  
}
```

Código 8 - Função IniciarDFS

4.4 Procura em largura BFS (Problema 3.b.)

A procura em largura (BFS – *Breadth-First Search*) é outro algoritmo clássico de travessia de grafos. Ao contrário da DFS, que explora os caminhos até ao fim antes de recuar, a BFS visita os vértices por camadas, isto é, explora primeiro todos os vizinhos diretos da antena de origem, depois os vizinhos dos vizinhos, e assim sucessivamente.

Neste projeto, a BFS foi utilizada para listar todas as antenas acessíveis a partir de uma determinada antena, respeitando a ordem de distância em saltos. Para isso, é utilizada uma estrutura auxiliar de fila, garantindo que os nós são processados na ordem correta.

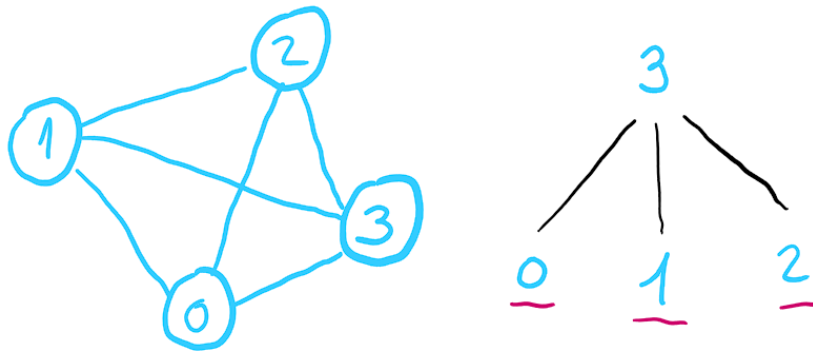


Figura 8 - BFS – *Breadth-First Search*

4.4.1 Função BFS

A função `BFS()` implementa a lógica da travessia em largura, utilizando três estruturas:

- Um vetor `visitado[]`, para evitar repetições;
- Um vetor `fila[]`, para controlar a ordem de exploração;
- Um vetor `origem[]`, que guarda de onde veio cada antena (para efeitos de rastreio e impressão).

A função imprime, para cada antena visitada, a sua posição, frequência, e a origem da aresta pela qual foi alcançada.

```
int BFS(Grafo* grafo, int idOrigem) {
    if (grafo->Antena[idOrigem] == NULL) {
        printf("Antena com ID %d não existe.\n", idOrigem);
        return 200;
    }
    int visitado[MAX_VERTICES] = { 0 };
    int fila[MAX_VERTICES];
    int origem[MAX_VERTICES];
    int frente = 0, tras = 0;
    fila[tras] = idOrigem;
    origem[tras] = -1;
    visitado[idOrigem] = 1;
    printf("BFS a partir da antena %d:\n", idOrigem);
    while (frente <= tras) {
        int atualID = fila[frente];
        int origemID = origem[frente];
        frente++;
        Ant* atual = grafo->Antena[atualID];
        if (origemID == -1) {
            printf("Antena %d (%c) em [%d, %d] (Origem)\n",
                atual->id, atual->freqAntena, atual->x, atual->y);
        }
        else {
            printf("Antena %d (%c) em [%d, %d] (Aresta: origem %d -> destino %d)\n",
                atual->id, atual->freqAntena, atual->x, atual->y, origemID, atualID);
        }
        Ars* aresta = atual->listaAresta;
        while (aresta != NULL) {
            int vizinhoID = aresta->destinoAntena->id;
            if (!visitado[vizinhoID]) {
                tras++;
                fila[tras] = vizinhoID;
                origem[tras] = atualID;
                visitado[vizinhoID] = 1;
            }
            aresta = aresta->proximaAresta;
        }
    }
    printf("\n\n");
    return 200;
}
```

Código 9 - Função BFS

4.5 Caminhos possíveis entre antena (Problemas 3.c.)

Um dos objetivos propostos para esta fase do projeto foi a identificação de todos os caminhos possíveis entre duas antenas. Esta funcionalidade é particularmente útil para analisar a conectividade entre dispositivos da mesma frequência e pode ser usada para validar a existência de redundância ou caminhos alternativos em redes de comunicação.

A estratégia utilizada baseia-se numa exploração em profundidade com backtracking, garantindo que todos os caminhos válidos entre a antena de origem e a antena de destino são descobertos e listados.

4.5.1 Função EncontrarCaminhos

A função `EncontrarCaminhos()` é uma função recursiva que percorre o grafo a partir da antena atual e tenta chegar ao destino, mantendo um vetor `caminho[]` para registar o percurso atual e um vetor `visitado[]` para evitar ciclos.

Quando o destino é alcançado, o caminho completo é impresso. Após cada chamada recursiva, o algoritmo recua (backtracking) para explorar alternativas.

```
int EncontrarCaminhos(Ant* atual, int idDestino, int visitado[], int caminho[], int
posicao) {
    if (atual == NULL) return 200;
    visitado[atual->id] = 1;
    caminho[posicao++] = atual->id;
    if (atual->id == idDestino) {
        for (int i = 0; i < posicao; i++) {
            printf("%d", caminho[i]);
            if (i < posicao - 1) printf(" -> ");
        }
        printf("\n");
    }
    else {
        Ars* aresta = atual->listaAresta;
        while (aresta != NULL) {
            int vizinhoID = aresta->destinoAntena->id;
            if (!visitado[vizinhoID]) {
                encontrarCaminhos(aresta->destinoAntena, idDestino, visitado, caminho,
posicao);
            }
            aresta = aresta->proximaAresta;
        }
    }
    visitado[atual->id] = 0;
    return 200;
}
```

Código 10 - Função EncontrarCaminhos

4.5.2 Função TodosCaminhos

A função TodosCaminhos() é o ponto de entrada para o utilizador. Ela valida os identificadores de origem e destino e prepara os vetores necessários (visitado[] e caminho[]) antes de chamar a função recursiva EncontrarCaminhos().

```
int TodosCaminhos(Grafo* grafo, int idOrigem, int idDestino) {
    if (grafo->Antena[idOrigem] == NULL || grafo->Antena[idDestino] == NULL) {
        printf("Antena(s) inválida(s)\n");
        return 200;
    }
    int visitado[MAX_VERTICES] = { 0 };
    int caminho[MAX_VERTICES];
    printf("Todos os caminhos de %d para %d:\n", idOrigem, idDestino);
    encontrarCaminhos(grafo->Antena[idOrigem], idDestino, visitado, caminho, 0);
    printf("\n\n");
    return 200;
}
```

Código 11 - Função TodosCaminhos

5. Repositório GitHub

O repositório *GitHub* com o projeto de Estruturas de Dados e Algoritmos (EDA), a sua documentação, o enunciado do projeto, e o seu relatório pode ser acedido através do seguinte link:

https://github.com/fabiocosta191/Projeto_EDA

Neste repositório, poderão ser consultados todos os detalhes e documentação relacionados com a primeira (antes e pós correções sugeridas na defesa) e segunda fase do projeto, incluindo as análises, implementações e conclusões até ao momento.

6. Considerações Finais

Ao longo da realização desta segunda fase do projeto prático, aprofundei significativamente a minha compreensão sobre a implementação e manipulação de grafos dinâmicos em linguagem C. A transição de estruturas simples, como listas ligadas (exploradas na Fase 1), para uma estrutura mais complexa baseada em listas de adjacência, representou um desafio importante, sobretudo ao nível da organização dos dados, da gestão eficiente da memória e da integração modular do código.

A implementação de operações como a criação de arestas entre antenas, o armazenamento de zonas nefastas, e a execução de algoritmos de travessia como DFS, BFS e backtracking, permitiu-me consolidar conhecimentos fundamentais sobre grafos e perceber como a estruturação correta dos dados influencia diretamente a eficiência e legibilidade da solução.

A necessidade de determinar todos os caminhos possíveis entre duas antenas destacou ainda mais a importância de raciocínios lógicos e estruturados, especialmente no uso de algoritmos recursivos com backtracking, reforçando a minha capacidade de resolver problemas com múltiplas ramificações e estados.

Este projeto contribuiu também para o reforço de boas práticas de desenvolvimento, tais como:

- A modularização do código;
- A documentação clara através do Doxygen;
- O controlo de versões com GitHub, essencial para o acompanhamento da evolução do trabalho.

De forma geral, considero que este projeto foi uma excelente oportunidade para aplicar e consolidar os conteúdos abordados na unidade curricular de Estruturas de Dados Avançadas, não só cumprindo os objetivos propostos, mas também me preparando para problemas mais complexos no âmbito da programação, da análise de algoritmos e da modelação de sistemas.

A experiência adquirida nesta fase será uma mais-valia para o meu percurso académico e profissional, dotando-me de ferramentas sólidas para abordar desafios relacionados com redes, estruturas interligadas e otimização de caminhos.

7. Referencias bibliográfica

- (43) *Linguagem C - Português - YouTube*. (n.d.). Retrieved March 29, 2025, from <https://www.youtube.com/@linguagemc-portugues1473/videos>
- (976) *Estrutura de Dados - Aula 23 - Grafos - Conceitos básicos - YouTube*. (n.d.). Retrieved May 12, 2025, from <https://www.youtube.com/watch?v=MC0u4f334mI>
- (976) *Estrutura de Dados em C | Aula 56 - Grafos - Definição - YouTube*. (n.d.). Retrieved May 12, 2025, from <https://www.youtube.com/watch?v=gJvSmrxeKDo>
- (976) *Estrutura de Dados em C | Aula 57 - Grafos - Propriedades - YouTube*. (n.d.). Retrieved May 12, 2025, from <https://www.youtube.com/watch?v=qvSbkbUkZjo>
- (976) *Estrutura de Dados em C | Aula 58 - Grafos - Tipos de Grafos - Parte 1 - YouTube*. (n.d.). Retrieved May 12, 2025, from <https://www.youtube.com/watch?v=5saF2Dg6sIc>
- fabiocosta191/Projeto_EDA: Repositório para arquivar o projeto de EDA*. (n.d.). Retrieved April 9, 2025, from https://github.com/fabiocosta191/Projeto_EDA
- Grafos — código das operações básicas em C. | by Paulo Martins | Medium*. (n.d.). Retrieved May 12, 2025, from https://medium.com/@paulomartins_10299/grafos-c%C3%B3digo-das-operac%C3%A7%C3%B5es-b%C3%A1sicas-e91d9a79e828
- Livro Algoritmos para Grafos (em linguagem C)*. (n.d.). Retrieved May 12, 2025, from https://www.ime.usp.br/~pf/algoritmos_para_grafos/
- O manual do iniciante em C: aprenda o básico sobre a linguagem de programação C em apenas algumas horas*. (n.d.). Retrieved March 29, 2025, from <https://www.freecodecamp.org/portuguese/news/o-manual-do-iniciante-em-c-aprenda-o-basico-sobre-a-linguagem-de-programacao-c-em-apenas-algumas-horas/>
- Para que serve o struct do C? O que significa a->b?* (n.d.). Retrieved March 29, 2025, from <https://www.ime.usp.br/~pf/algoritmos/aulas/stru.html>
- Programação C - Structs*. (n.d.). Retrieved March 29, 2025, from <https://www.inf.pucrs.br/~pinho/Laprol/Structs/Structs.htm>
- Programação com apontadores*. (n.d.). Retrieved March 29, 2025, from <https://www.dcc.fc.up.pt/~pbv/aulas/progimp/teoricas/teorica23.html>